Programmation Orientée Objet (POO)

💡 Concept et Définition

La **POO** (Programmation Orientée Objet) est un **paradigme de programmation** qui consiste à définir des "objets" informatiques. Au lieu de voir un programme comme une simple suite d'instructions, on le conçoit comme une interaction entre des entités autonomes.

L'analogie du Robot :

->**La Classe (Le Plan) :** C'est le schéma technique à l'usine. Il définit que tout robot *doit* avoir un nom et une batterie, et *sait* marcher. Ce n'est pas encore un robot réel, c'est un **nouveau type** de donnée que vous créez.

->**L'Objet ou Instance (Le Robot réel) :** C'est l'unité qui sort de l'usine. On peut créer des "jumeaux" issus du même plan : deux robots identiques au départ, mais qui mèneront leur propre vie.

->**L'Attribut (Caractéristique) :** Ce sont les propriétés du robot (son nom, son niveau de batterie). Même s'ils ont le même constructeur, un robot peut être à 100% de batterie et l'autre à 10%.

->**La Méthode (Action) :** Ce sont les capacités du robot (marcher, saluer). C'est une fonction interne à l'objet.

->**Le Constructeur (L'assemblage) :** C'est l'étape de fabrication qui donne ses caractéristiques initiales au robot au moment où il "naît".

📊 Les Attributs : Accès et Modification

Un **attribut** est une variable interne à l'objet qui définit son état. On utilise la notation pointée `.` pour agir dessus. On trouve usuellement les attributs dans le constructeur (__init__).

Accès :

```
root@python:~$
# Accès à un attribut
print(mon_robot.nom)  # Affiche "R2D2"
```

Modification :

```
root@python:~$
# Modification d'un attribut
mon_robot.v = 2.0
print(mon_robot.v) # Affiche 2.0
```

⚙️ Les Méthodes et leurs Appels

Une **méthode** est une fonction définie à l'intérieur d'une classe. Elle représente un comportement de l'objet. Le paramètre `self` permet à la méthode d'accéder aux attributs de l'objet qui l'appelle.

Définition de méthode :

```
root@python:~$
class Robot:
    def __init__(self, nom, version=1.0):
        # Initialisation des attributs (état de l'objet)
        self.nom = nom        # Passé à l'assemblage
        self.v = version      # Valeur par défaut
        self.batt = 100       # Valeur fixe au départ

    def saluer(self): # Définition d'une méthode
        print(f"Bonjour, je suis {self.nom}")

    def charger(self, energie):
        self.batt += energie
```

Appel de méthode :

Voyons maintenant un exemple d'appel de la méthode.

```
root@python:~$
# Appel de méthode
mon_robot.saluer()       # Affiche "Bonjour, je suis R2D2"
mon_robot.charger(20)    # Modifie l'état interne (batterie)
```

🏗️ Le Constructeur

En Python, le constructeur s'appelle `__init__`, il permet de donner une valeur aux attributs d'un objet. Il est appelé à la création de chaque objet. C'est ici que l'on transforme le plan en un objet concret avec ses propres valeurs.

```
root@python:~$
class Robot:
    def __init__(self, nom, version=1.0):
        # Initialisation des attributs (état de l'objet)
        self.nom = nom        # Passé à l'assemblage
        self.v = version      # Valeur par défaut
        self.batt = 100       # Valeur fixe au départ
```

Contrairement aux autres méthodes, on ne l'appelle jamais manuellement (on n'écrit pas `robot.__init__()`) : Python l'exécute **automatiquement** en arrière-plan au moment précis où l'on crée l'objet (en utilisant le nom de la classe `Robot()`) pour garantir qu'il naisse dans un état valide.

🏗️ Création d'objet (Instanciation)

Pour créer un objet on utilise **le nom de la classe** comme si c'était une fonction. La fonction véritablement appelée est `__init__` mais on ne l'appelle jamais explicitement.

Création d'objet :

Création de "jumeaux" (Appel automatique de __init__).

```
root@python:~$
robot_A = Robot("R2D2")
robot_B = Robot("R2D2")
robot_C = Robot("C3PO", 4.0)
```

Indépendance des objets :

Ils sont du même type (Robot) mais sont des objets distincts.

```
root@python:~$
robot_A.batt = 20  # Robot_A est déchargé, mais pas Robot_B !
print(robot_A.batt) # Affiche 20
print(robot_B.batt) # Affiche 100
```

🆔 Comprendre le mot-clé "self"

Self représente l'instance elle-même (le "moi"). C'est le lien entre le code générique de la classe et l'objet spécifique en mémoire.

->**Pourquoi est-il partout ?** Dans la classe, Python ne sait pas encore quel robot va appeler la méthode. `self` dit : "Applique cette action sur l'objet qui m'a appelé".

->**Le paramètre invisible :** Il doit être le **premier argument** de chaque méthode dans la classe, mais on ne lui passe **jamais** de valeur lors de l'appel.

```
root@python:~$
def afficher_nom(self):
    # Sans "self.", Python chercherait une variable locale
    # Avec "self.", il va chercher l'attribut DANS l'objet.
    print(f"Mon nom est {self.nom}")
```

Self n'est jamais utilisé en dehors de la classe (en dehors des méthodes).

```
root@python:~$
robot_A.afficher_nom()
# Python fait secrètement : afficher_nom(robot_A)
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0